

Java Reference Types – Using String as the Most Common Reference Type

1. What is a Reference Type?

A **reference type** is a type whose variable **does not store the actual data**. Instead, it stores the **memory address (reference)** of an object that exists in the **Heap Memory**.

Think of it like this:

- Primitive variable → stores the actual value.
- Reference variable → stores the address of an object.

Example:

```
String name = "Siraj";
```

Here:

- name is a reference variable.
 - "Siraj" is a String object.
 - name stores the reference (address) to that object.
-

2. Why is String a Reference Type?

Although String feels simple like int, it is actually a **class**.

```
String city = "Lahore";
```

This is actually shorthand for:

```
String city = new String("Lahore");
```

except when using a string literal, Java optimizes it by storing it in the **String Pool**.

Since String is a class, every String value is an **object**.

Objects always live in the **Heap**.

Variables referring to those objects live on the **Stack** (for local variables).

3. Primitive Types vs Reference Types

Primitive Type	Reference Type
Stores actual value	Stores memory reference
Stored directly	Refers to an object
Fixed size	Size depends on object
Cannot be null	Can be null
Faster access	Slightly slower due to indirection
Examples: int, char, boolean	Examples: String, Arrays, Classes, Interfaces

Example:

```
int age = 22;
```

Memory:

```
Stack
-----
age = 22
```

Reference example:

```
String name = "Siraj";
```

Memory:

Stack	Heap
-----	-----
name ----->	"Siraj"

4. Why Use String to Learn Reference Types?

Most Java programs use String extensively.

Examples:

```
Scanner scanner = new Scanner(System.in);  
  
String name = scanner.nextLine();  
  
System.out.println(name);
```

Every username, password, filename, URL, JSON key, and API response often involves String.

That makes it the most common reference type beginners encounter.

5. Creating Strings

Method 1 (Recommended)

```
String language = "Java";
```

Uses the **String Pool**.

Method 2

```
String language = new String("Java");
```

Always creates a new object in the Heap.

Even if "Java" already exists in the pool, new creates another object.

6. How String Objects Are Stored

Example:

```
String s1 = "Java";  
String s2 = "Java";
```

Memory:

```
Stack  
  
s1 ----\  
      \  
      ---> "Java" (String Pool)  
      /  
s2 ----/
```

Only **one object** exists.

Both variables point to the same object.

Example:

```
String s3 = new String("Java");
```

Memory:

Stack

s3 -----> Heap Object

String Pool -----> "Java"

Now two objects exist:

- pooled "Java"
- heap "Java"

7. What Does the Reference Variable Actually Store?

Many beginners think:

```
String name = "Ali";
```

stores

```
name = Ali
```

Wrong.

Actually:

```
name = 0xA13F28
```

where 0xA13F28 represents a memory reference (illustrative only; Java does not expose actual addresses).

The JVM hides real memory addresses.

8. null Reference

Reference variables can point to nothing.

```
String name = null;
```

Memory:

```
Stack  
  
name -----> null
```

Trying to call a method:

```
name.length();
```

throws:

```
NullPointerException
```

9. Reassigning References

```
String s = "Java";  
  
s = "Python";
```

Memory:

```
Initially  
  
Stack
```

```
s ----> "Java"
```

After assignment

Stack

```
s ----> "Python"
```

The "Java" object still exists until it becomes unreachable. If no references point to it, it becomes eligible for **Garbage Collection**.

10. String Immutability

Strings cannot be modified after creation.

```
String s = "Java";  
  
s.concat(" Programming");
```

Result?

Nothing changes.

Need:

```
s = s.concat(" Programming");
```

Now:

Old Object

"Java"

New Object

11. String Pool

The JVM maintains a special memory area called the **String Pool** to avoid duplicate string literals.

Example:

```
String a = "Hello";  
String b = "Hello";  
String c = "Hello";
```

Only one object exists.

Benefits:

- Saves memory
 - Faster comparisons using == (for pooled literals)
 - Better performance
-

12. == vs equals()

This is one of the most frequently asked Java interview questions.

```
String a = "Java";  
String b = "Java";  
  
System.out.println(a == b);
```

Output:

```
true
```


Because both references point to the same pooled object.

```
String a = new String("Java");  
String b = new String("Java");  
  
System.out.println(a == b);
```

Output:

```
false
```

Different objects.

Correct comparison:

```
a.equals(b)
```

Output:

```
true
```

`equals()` compares **content**, not references.

13. Stack vs Heap

```
String name = "Siraj";
```

```
STACK
```

```
name  
|
```

|
v

HEAP (String Pool)

"Siraj"

- Variable → Stack (for local variables)
 - Object → Heap
 - Reference connects them
-

14. Common Beginner Mistakes

- ✗ Thinking String is primitive.
 - ✗ Thinking variable stores characters directly.
 - ✗ Using == to compare text.
 - ✗ Forgetting Strings are immutable.
 - ✗ Believing new String() uses the String Pool object directly.
-

15. Real-World Examples

```
String username = "admin";  
String password = "12345";  
String fileName = "report.pdf";  
String url = "https://example.com";  
String json = "{\"id\":1}";
```

All are references to String objects.

16. Best Practices

- ✓ Prefer:

```
String name = "Ali";
```

instead of:

```
String name = new String("Ali");
```

unless you specifically need a distinct object.

Use:

```
equals()
```

instead of:

```
==
```

for comparing text.

Avoid:

```
String s = null;
```

unless null has a clear semantic meaning and you handle it safely.

17. Tricky Interview Questions

Q1. Is String a primitive type?

Answer: No. It is a class in the java.lang package.

Q2. Where is a String object stored?

Answer: In the Heap. String literals are additionally managed in the String Pool, which is part of the Heap in modern JVM implementations.

Q3. Where is the reference variable stored?

Answer: A local reference variable is stored on the **Stack**, while the object it points to is on the **Heap**.

Q4. Does a reference variable contain the object?

Answer: No. It contains a reference to the object.

Q5. Can a reference variable be null?

Answer: Yes.

```
String s = null;
```

Q6. What happens if we call a method on null?

Answer: A NullPointerException is thrown.

Q7. Is String mutable?

Answer: No. It is immutable.

Q8. Why is String immutable?

Answer:

- Security
 - Thread safety
 - String Pool optimization
 - Hash code caching
 - Better performance in many scenarios
-

Q9. Why is String the most used reference type?

Answer: Nearly every Java application handles text—user input, files, URLs, JSON, XML, SQL queries, logs, configuration, and API data all rely heavily on String.

Q10. What does a reference variable actually store?

Answer: A reference to an object (conceptually an address), not the object's contents.

18. Output-Based Interview Questions

Question 1

```
String a = "Java";  
String b = "Java";  
  
System.out.println(a == b);
```

Output:

```
true
```

Question 2

```
String a = new String("Java");  
String b = new String("Java");  
  
System.out.println(a == b);
```

Output:

```
false
```

Question 3

```
String a = new String("Java");  
String b = new String("Java");  
  
System.out.println(a.equals(b));
```

Output:

```
true
```

Question 4

```
String a = null;  
  
System.out.println(a.length());
```

Output:

```
NullPointerException
```

Question 5

```
String a = "Java";  
  
a.concat(" Programming");  
  
System.out.println(a);
```

Output:

```
Java
```

Reason: concat() returns a new String; it does not modify the original.

19. Interview Cheat Sheet

- String is a **reference type**, not a primitive.
 - String is a **class** in java.lang.
 - Reference variables store **references**, not object data.
 - Objects live in the **Heap**.
 - Local reference variables live on the **Stack**.
 - String literals are stored in the **String Pool**.
 - `==` compares references.
 - `equals()` compares contents.
 - String is immutable.
 - `null` means "no object is being referenced."
 - Prefer string literals ("text") over `new String("text")` unless you specifically need a new object.
-

Top 10 Interview Questions to Memorize

1. Is String a primitive or reference type?
2. Why is String immutable?
3. What is the String Pool?
4. What is the difference between `==` and `equals()`?
5. Where are String objects stored?
6. Where is the reference variable stored?
7. What does `new String("Java")` create?
8. Can a reference variable be null?
9. Why are string literals reused?
10. Explain the memory layout of `String s = "Java";` using Stack and Heap.

These concepts form the foundation for understanding Java object references and are asked frequently in internship, fresher, and backend Java interviews.